Precompiled Contracts 완전 가이드

목차

1. Precompiled Contracts 개요
2. 표준 Ethereum Precompiles
3. Cosmos EVM Custom Precompiles
4. Precompile 구현 방법
5. Gas 비용 설계
6. 테스트 및 검증
7. 실전 예제
8. 보안 고려사항
9. 디버깅 및 모니터링
10. Best Practices

---

1. Precompiled Contracts 개요

1.1 Precompile이란?

Precompiled Contracts는 EVM에서 특별한 주소에 미리 컴파일되어 내장된 네이티브 함수들입니다.

주요 특징

- **고성능**: Solidity 대신 Go/Rust 등으로 구현되어 훨씬 빠름
- **저렴한 Gas**: 최적화된 구현으로 Gas 비용 절감
- **특수 기능**: EVM만으로 불가능한 암호화 연산 등 제공
- **고정 주소**: `0x01` ~ `0x09` 등 예약된 주소 사용

왜 필요한가?

```
// ❌ Solidity로 SHA256 구현 시 - 매우 비효율적
function sha256Slow(bytes memory data) public pure returns (bytes32) {
    // 수백 줄의 비트 연산 코드...
    // Gas 소모: ~50,000 Gas
}

// ✅ Precompile 사용 시 - 네이티브 함수 호출
function sha256Fast(bytes memory data) public pure returns (bytes32) {
    return sha256(data); // 0x02 주소 호출
    // Gas 소모: ~60 Gas + 12 Gas/word
}
```

1.2 Precompile의 작동 원리

```
┌──────────────────────────────────────────────────────┐
│                    Smart Contract                    │
│  function example() {                                │
│      address(0x02).call(data); // SHA256 precompile  │
│  }                                                   │
└──────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
┌──────────────────────────────────────────────────────┐
│                    EVM Interpreter                   │
│                                                      │
│  if (address == 0x02) {                              │
│      return precompiledContracts[0x02].Run(input);   │
│  }                                                   │
└──────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
┌──────────────────────────────────────────────────────┐
│              Native Go Implementation                │
│                                                      │
│  func (c *sha256hash) Run(input []byte) ([]byte, error) │
│  {                                                   │
│      h := sha256.Sum256(input)                       │
│      return h[:], nil                                │
│  }                                                   │
└──────────────────────────────────────────────────────┘
```

---

2. 표준 Ethereum Precompiles

2.1 EVM 기본 Precompiles

주소	이름	기능	Gas 비용
0x01	ECRECOVER	ECDSA 서명 검증	3,000 Gas
0x02	SHA256	SHA-256 해시	60 + 12/word
0x03	RIPEMD160	RIPEMD-160 해시	600 + 120/word
0x04	IDENTITY	데이터 복사	15 + 3/word
0x05	MODEXP	모듈러 지수 연산	동적
0x06	ECADD	BN256 타원곡선 덧셈	500 Gas
0x07	ECMUL	BN256 타원곡선 곱셈	40,000 Gas
0x08	ECPAIRING	BN256 페어링 검사	100,000 + k*80,000
0x09	BLAKE2F	BLAKE2b F 압축	동적

2.2 ECRECOVER (0x01) 상세 분석

가장 많이 사용되는 precompile로, 서명으로부터 주소를 복구합니다.

```go
// go-ethereum/core/vm/contracts.go
type ecrecover struct{}

func (c *ecrecover) RequiredGas(input []byte) uint64 {
    return params.EcrecoverGas // 3000 Gas
}

func (c *ecrecover) Run(input []byte) ([]byte, error) {
    const ecRecoverInputLength = 128

    // Input 검증
    input = common.RightPadBytes(input, ecRecoverInputLength)

    // 32 bytes: hash
    // 32 bytes: v
    // 32 bytes: r
    // 32 bytes: s

    r := new(big.Int).SetBytes(input[64:96])
    s := new(big.Int).SetBytes(input[96:128])
    v := input[63] - 27

    // Signature validation
    if !crypto.ValidateSignatureValues(v, r, s, false) {
        return nil, nil
    }

    // Public key 복구
    pubKey, err := crypto.Ecrecover(input[:32], append(input[64:128], v))
    if err != nil {
        return nil, nil
    }

    // Address 추출 (pubkey의 Keccak256 마지막 20 bytes)
    return common.LeftPadBytes(crypto.Keccak256(pubKey[1:])[12:], 32), nil
}
```

Solidity에서 사용하기

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SignatureVerifier {
    // Method 1: Built-in ecrecover 함수 사용
    function recoverSigner(
        bytes32 hash,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) public pure returns (address) {
        return ecrecover(hash, v, r, s);
    }

    // Method 2: Low-level call 사용
    function recoverSignerLowLevel(
        bytes32 hash,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) public view returns (address) {
        bytes memory input = abi.encodePacked(hash, v, r, s);

        (bool success, bytes memory result) = address(0x01).staticcall(input);
        require(success, "ecrecover failed");

        return address(uint160(uint256(bytes32(result))));
    }

    // 실전 예제: EIP-712 서명 검증
    function verifySignature(
        address signer,
        bytes32 structHash,
        bytes memory signature
    ) public view returns (bool) {
        require(signature.length == 65, "Invalid signature length");

        bytes32 r;
        bytes32 s;
        uint8 v;

        assembly {
            r := mload(add(signature, 32))
            s := mload(add(signature, 64))
            v := byte(0, mload(add(signature, 96)))
        }

        address recovered = ecrecover(structHash, v, r, s);
        return recovered == signer;
    }
}
```

2.3 SHA256 (0x02) & RIPEMD160 (0x03)

```go
// SHA256 구현
type sha256hash struct{}

func (c *sha256hash) RequiredGas(input []byte) uint64 {
    return uint64(len(input)+31)/32*params.Sha256PerWordGas + params.Sha256BaseGas
    // 기본 60 Gas + (워드당 12 Gas)
}

func (c *sha256hash) Run(input []byte) ([]byte, error) {
    h := sha256.Sum256(input)
    return h[:], nil
}
```

Bitcoin 주소 생성에 사용

```
contract BitcoinAddressGenerator {
    // Bitcoin 주소 = Base58(RIPEMD160(SHA256(pubkey)))
    function generateBitcoinAddress(bytes memory pubkey)
        public
        view
        returns (bytes20)
    {
        // Step 1: SHA256
        bytes32 sha = sha256(pubkey);

        // Step 2: RIPEMD160
        bytes20 hash;
        (bool success, bytes memory result) = address(0x03).staticcall(
            abi.encodePacked(sha)
        );
        require(success);

        assembly {
            hash := mload(add(result, 32))
        }

        return hash;
    }
}
```

2.4 BN256 Precompiles (0x06, 0x07, 0x08)

zk-SNARKs 검증에 필수적인 타원곡선 연산

```
// Groth16 zk-SNARK 검증 예제
contract ZKVerifier {
    struct Proof {
        uint[2] a;
        uint[2][2] b;
        uint[2] c;
    }

    struct VerifyingKey {
        uint[2] alpha;
        uint[2][2] beta;
        uint[2][2] gamma;
        uint[2][2] delta;
        uint[2][] ic; // Input commitments
    }

    function verify(
        Proof memory proof,
        VerifyingKey memory vk,
        uint[] memory input
    ) public view returns (bool) {
        // Pairing check: e(A, B) = e(alpha, beta) * e(C, delta) * e(IC, gamma)

        uint[24] memory pairingInput;

        // 0x08 (ECPAIRING) precompile 호출
        (bool success, bytes memory result) = address(0x08).staticcall(
            abi.encodePacked(pairingInput)
        );

        require(success, "Pairing check failed");
        return abi.decode(result, (bool));
    }
}
```

3. Cosmos EVM Custom Precompiles

3.1 왜 Custom Precompiles가 필요한가?

Cosmos SDK 기능을 EVM 스마트 컨트랙트에서 사용하기 위해:

```
┌──────────────────────────────────────────┐
│         EVM Smart Contract (Solidity)    │
│                                          │
│  - 토큰 전송하고 싶다                    │
│  - Validator에 위임하고 싶다             │
│  - IBC 패킷 보내고 싶다                  │
└──────────────────────────────────────────┘
                  │
                  │
                  │ ❌ 불가능! EVM은 Cosmos SDK 모르는데?
                  │
                  ▼
┌──────────────────────────────────────────┐
│         ✅ Custom Precompiles 솔루션     │
│                                          │
│  address(0x0800).call("delegate", validator)  │
│          ↓                               │
│  Cosmos SDK staking module 호출          │
└──────────────────────────────────────────┘
```

3.2 Cosmos EVM 아키텍처

```go
// cosmos/evm/x/evm/vm/precompiles.go
var CosmosPrecompiles = map[common.Address]vm.PrecompiledContract{
    // Cosmos SDK 모듈 접근
    common.BytesToAddress([]byte{0x08, 0x00}): &stakingPrecompile{},
    common.BytesToAddress([]byte{0x08, 0x01}): &bankPrecompile{},
    common.BytesToAddress([]byte{0x08, 0x02}): &govPrecompile{},
    common.BytesToAddress([]byte{0x08, 0x03}): &distributionPrecompile{},
    common.BytesToAddress([]byte{0x08, 0x04}): &ibcPrecompile{},

    // 추가 암호화 기능
    common.BytesToAddress([]byte{0x08, 0x10}): &bls12381Precompile{},
    common.BytesToAddress([]byte{0x08, 0x11}): &ed25519Precompile{},
}
```

---

4. Precompile 구현 방법

4.1 PrecompiledContract 인터페이스

```go
// core/vm/contracts.go
type PrecompiledContract interface {
    // Gas 비용 계산
    RequiredGas(input []byte) uint64

    // 실제 실행 로직
    Run(input []byte) ([]byte, error)
}
```

4.2 단계별 구현 가이드

Step 1: 구조체 정의

```go
package precompiles

import (
    "github.com/ethereum/go-ethereum/common"
    "github.com/ethereum/go-ethereum/core/vm"
    sdk "github.com/cosmos/cosmos-sdk/types"
)

// Staking precompile 구현
type stakingPrecompile struct {
    stakingKeeper StakingKeeperInterface
}

// StakingKeeperInterface - Cosmos SDK와의 연결
type StakingKeeperInterface interface {
    Delegate(ctx sdk.Context, delAddr sdk.AccAddress,
             valAddr sdk.ValAddress, amount sdk.Coin) error
    Undelegate(ctx sdk.Context, delAddr sdk.AccAddress,
               valAddr sdk.ValAddress, amount sdk.Dec) error
    GetDelegation(ctx sdk.Context, delAddr sdk.AccAddress,
                  valAddr sdk.ValAddress) (sdk.Delegation, bool)
}
```

Step 2: Gas 비용 구현

```go
func (p *stakingPrecompile) RequiredGas(input []byte) uint64 {
    // Input parsing (첫 4 bytes는 function selector)
    if len(input) < 4 {
        return 0
    }

    methodID := input[:4]

    switch string(methodID) {
    case "dele": // delegate
        // StateDB 읽기/쓰기 비용 고려
        return 50000 // 기본 50k Gas

    case "unde": // undelegate
        return 50000

    case "getD": // getDelegation (읽기 전용)
        return 5000 // 읽기는 저렴

    default:
        return 0
    }
}
```

Step 3: 실행 로직 구현

```go
func (p *stakingPrecompile) Run(input []byte) ([]byte, error) {
    // 1. Input 파싱
    if len(input) < 4 {
        return nil, vm.ErrInvalidInput
    }

    methodID := input[:4]
    args := input[4:]

    // 2. Method 라우팅
    switch string(methodID) {
    case "dele":
        return p.delegate(args)
    case "unde":
        return p.undelegate(args)
    case "getD":
        return p.getDelegation(args)
    default:
        return nil, vm.ErrInvalidMethod
    }
}

// Delegate 구현
func (p *stakingPrecompile) delegate(input []byte) ([]byte, error) {
    // ABI 디코딩
    type DelegateArgs struct {
        ValidatorAddr common.Address // 20 bytes
        Amount        *big.Int        // 32 bytes
    }

    if len(input) < 52 {
        return nil, vm.ErrInvalidInput
    }

    args := DelegateArgs{
        ValidatorAddr: common.BytesToAddress(input[:20]),
        Amount:        new(big.Int).SetBytes(input[20:52]),
    }

    // Cosmos SDK Context 가져오기
    ctx := sdk.UnwrapSDKContext(evm.GetContext())

    // EVM caller를 Cosmos address로 변환
    delAddr := sdk.AccAddress(evm.Caller.Bytes())
    valAddr := sdk.ValAddress(args.ValidatorAddr.Bytes())

    // Staking module 호출
    coin := sdk.NewCoin("stake", sdk.NewIntFromBigInt(args.Amount))
    if err := p.stakingKeeper.Delegate(ctx, delAddr, valAddr, coin); err != nil {
        return nil, err
    }

    // Success 반환
    return []byte{0x01}, nil
}
```

4.3 완전한 예제: BLS Signature Precompile

```go
// BLS12-381 서명 검증 precompile
type bls12381Precompile struct{}

func (p *bls12381Precompile) RequiredGas(input []byte) uint64 {
    // BLS 검증은 비용이 높음
    return 150000 // 150k Gas
}

func (p *bls12381Precompile) Run(input []byte) ([]byte, error) {
    // Input format:
    // [0:48]   - public key (48 bytes)
    // [48:144] - signature (96 bytes)
    // [144:]   - message

    if len(input) < 144 {
        return nil, vm.ErrInvalidInput
    }

    pubKeyBytes := input[0:48]
    signatureBytes := input[48:144]
    message := input[144:]

    // BLS 라이브러리 사용 (예: herumi/bls)
    pubKey := new(bls.PublicKey)
    if err := pubKey.Deserialize(pubKeyBytes); err != nil {
        return nil, err
    }

    signature := new(bls.Sign)
    if err := signature.Deserialize(signatureBytes); err != nil {
        return nil, err
    }

    // 서명 검증
    if signature.Verify(pubKey, message) {
        return common.LeftPadBytes([]byte{0x01}, 32), nil
    }

    return common.LeftPadBytes([]byte{0x00}, 32), nil
}
```

Solidity 인터페이스

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IBLS {
    function verify(
        bytes calldata publicKey,   // 48 bytes
        bytes calldata signature,   // 96 bytes
        bytes calldata message
    ) external view returns (bool);
}

contract BLSVerifier {
    address constant BLS_PRECOMPILE = 0x0000000000000000000000000000000000000810;

    function verifyBLS(
        bytes memory pubkey,
        bytes memory sig,
        bytes memory msg
    ) public view returns (bool) {
        bytes memory input = abi.encodePacked(pubkey, sig, msg);

        (bool success, bytes memory result) = BLS_PRECOMPILE.staticcall(input);
        require(success, "BLS precompile failed");

        return abi.decode(result, (bool));
    }
}
```

5. Gas 비용 설계

5.1 Gas 비용 고려사항

```go
// Gas 비용 = 기본 비용 + 계산 비용 + 상태 접근 비용

func calculateGas(operation string, dataSize int) uint64 {
    var baseGas uint64

    switch operation {
    case "hash":
        // CPU 집약적
        baseGas = 60
        return baseGas + uint64(dataSize)/32 * 12

    case "signature_verify":
        // 암호화 연산
        baseGas = 3000
        return baseGas

    case "state_read":
        // StateDB 읽기
        baseGas = 2100 // SLOAD
        return baseGas

    case "state_write":
        // StateDB 쓰기
        baseGas = 20000 // SSTORE (0 → non-zero)
        return baseGas

    case "complex_computation":
        // 벤치마크 기반으로 결정
        baseGas = 50000
        return baseGas + uint64(dataSize) * 10
    }

    return 0
}
```

5.2 벤치마킹 예제

```go
func BenchmarkStakingPrecompile(b *testing.B) {
    // Setup
    p := &stakingPrecompile{
        stakingKeeper: mockStakingKeeper,
    }

    input := prepareDelegateInput()

    // Benchmark
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        _, _ = p.Run(input)
    }

    // 결과 분석
    // BenchmarkStakingPrecompile-8  50000  35000 ns/op
    // → 35µs per operation
    // → Ethereum block time: 12s
    // → Gas limit: 30M
    // → 적절한 Gas: 35µs / (12s / 30M) ≈ 87,500 Gas
}
```

6. 테스트 및 검증

6.1 단위 테스트

```go
func TestStakingPrecompile_Delegate(t *testing.T) {
    tests := []struct {
        name    string
        input   []byte
        wantErr bool
        check   func(t *testing.T, output []byte)
    }{
        {
            name: "성공적인 위임",
            input: prepareInput(
                "dele", // method
                validatorAddr,
                big.NewInt(1000000),
            ),
            wantErr: false,
            check: func(t *testing.T, output []byte) {
                require.Equal(t, []byte{0x01}, output)
            },
        },
        {
            name: "잘못된 validator 주소",
            input: prepareInput(
                "dele",
                invalidAddr,
                big.NewInt(1000000),
            ),
            wantErr: true,
        },
        {
            name: "금액 0",
            input: prepareInput(
                "dele",
                validatorAddr,
                big.NewInt(0),
            ),
            wantErr: true,
        },
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            p := setupPrecompile(t)
            output, err := p.Run(tt.input)

            if tt.wantErr {
                require.Error(t, err)
            } else {
                require.NoError(t, err)
                if tt.check != nil {
                    tt.check(t, output)
                }
            }
        })
    }
}
```

6.2 통합 테스트 (Solidity)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";

contract StakingPrecompileTest is Test {
    address constant STAKING = 0x0000000000000000000000000000000000000800;
    address validator = address(0x1234);

    function testDelegate() public {
        bytes memory input = abi.encodeWithSignature(
            "delegate(address,uint256)",
            validator,
            1 ether
        );

        (bool success, bytes memory result) = STAKING.call(input);
        assertTrue(success);
        assertEq(result, hex"01");
    }

    function testDelegateInsufficientBalance() public {
        vm.deal(address(this), 0.5 ether);

        bytes memory input = abi.encodeWithSignature(
            "delegate(address,uint256)",
            validator,
            1 ether
        );

        (bool success, ) = STAKING.call(input);
        assertFalse(success);
    }

    function testGasCost() public {
        uint256 gasBefore = gasleft();

        bytes memory input = abi.encodeWithSignature(
            "delegate(address,uint256)",
            validator,
            1 ether
        );
        STAKING.call(input);

        uint256 gasUsed = gasBefore - gasleft();

        // Gas 비용이 예상 범위 내인지 확인
        assertLt(gasUsed, 60000);
        assertGt(gasUsed, 40000);
    }
}
```

7. 실전 예제

7.1 IBC Transfer Precompile

```go
// IBC 토큰 전송을 EVM에서 가능하게
type ibcTransferPrecompile struct {
    ibcKeeper IBCKeeperInterface
}

func (p *ibcTransferPrecompile) Run(input []byte) ([]byte, error) {
    // Input: channel_id, receiver, amount, denom, timeout
    type IBCTransferArgs struct {
        ChannelID string
        Receiver  string
        Amount    *big.Int
        Denom     string
        Timeout   uint64
    }

    args := parseIBCArgs(input)

    ctx := sdk.UnwrapSDKContext(evm.GetContext())
    sender := sdk.AccAddress(evm.Caller.Bytes())

    // IBC Transfer 실행
    packet := ibctransfertypes.NewMsgTransfer(
        "transfer",
        args.ChannelID,
        sdk.NewCoin(args.Denom, sdk.NewIntFromBigInt(args.Amount)),
        sender.String(),
        args.Receiver,
        clienttypes.NewHeight(0, args.Timeout),
        0,
    )

    if err := p.ibcKeeper.Transfer(ctx, packet); err != nil {
        return nil, err
    }

    return []byte{0x01}, nil
}
```

Solidity 사용 예제

```
contract CrossChainBridge {
    address constant IBC = 0x0000000000000000000000000000000000000804;

    event IBCTransferInitiated(
        string channelID,
        address sender,
        string receiver,
        uint256 amount
    );

    function sendToOsmosis(
        string memory receiver,
        uint256 amount
    ) external payable {
        require(msg.value == amount, "Amount mismatch");

        bytes memory input = abi.encode(
            "channel-0",    // Osmosis channel
            receiver,       // cosmos1xxx...
            amount,
            "uatom",
            block.timestamp + 1 hours
        );

        (bool success, ) = IBC.call{value: amount}(input);
        require(success, "IBC transfer failed");

        emit IBCTransferInitiated("channel-0", msg.sender, receiver, amount);
    }
}
```

7.2 Governance Voting Precompile

```
type govPrecompile struct {
    govKeeper GovKeeperInterface
}

func (p *govPrecompile) Run(input []byte) ([]byte, error) {
    // Method: vote(proposalID, option, metadata)

    type VoteArgs struct {
        ProposalID uint64
        Option     int32  // 1=Yes, 2=Abstain, 3=No, 4=NoWithVeto
        Metadata   string
    }

    args := parseVoteArgs(input)

    ctx := sdk.UnwrapSDKContext(evm.GetContext())
    voter := sdk.AccAddress(evm.Caller.Bytes())

    // Vote 제출
    voteOption := govtypes.VoteOption(args.Option)
    if err := p.govKeeper.AddVote(
        ctx,
        args.ProposalID,
        voter,
        govtypes.NewNonSplitVoteOption(voteOption),
        args.Metadata,
    ); err != nil {
        return nil, err
    }

    return []byte{0x01}, nil
}
```

8. 보안 고려사항

8.1 입력 검증

```go
func validateInput(input []byte) error {
    // 1. 길이 검증
    if len(input) < MinInputLength {
        return ErrInputTooShort
    }
    if len(input) > MaxInputLength {
        return ErrInputTooLong
    }

    // 2. 주소 검증
    if !isValidAddress(input[0:20]) {
        return ErrInvalidAddress
    }

    // 3. 숫자 범위 검증
    amount := new(big.Int).SetBytes(input[20:52])
    if amount.Cmp(MaxAmount) > 0 {
        return ErrAmountTooLarge
    }

    // 4. 문자열 검증 (UTF-8, 길이 제한)
    str := string(input[52:])
    if !utf8.ValidString(str) {
        return ErrInvalidUTF8
    }

    return nil
}
```

8.2 재진입 공격 방지

```go
type stakingPrecompile struct {
    stakingKeeper StakingKeeperInterface
    mu            sync.Mutex
    inProgress    map[common.Address]bool
}

func (p *stakingPrecompile) Run(input []byte) ([]byte, error) {
    caller := evm.Caller

    // 재진입 체크
    p.mu.Lock()
    if p.inProgress[caller] {
        p.mu.Unlock()
        return nil, ErrReentrancy
    }
    p.inProgress[caller] = true
    p.mu.Unlock()

    defer func() {
        p.mu.Lock()
        delete(p.inProgress, caller)
        p.mu.Unlock()
    }()

    // 실제 로직 실행
    return p.executeDelegate(input)
}
```

8.3 Gas Limit 공격 방지

```go
func (p *complexPrecompile) RequiredGas(input []byte) uint64 {
    // DoS 방지: 입력 크기에 비례하여 Gas 증가
    baseGas := uint64(50000)
    dataSize := uint64(len(input))

    // 최대 Gas 제한
    maxGas := uint64(10000000) // 10M Gas

    gas := baseGas + dataSize*100
    if gas > maxGas {
        return maxGas
    }

    return gas
}
```

---

9. 디버깅 및 모니터링

9.1 로깅

```go
func (p *stakingPrecompile) Run(input []byte) ([]byte, error) {
    logger := ctx.Logger().With(
        "module", "evm",
        "precompile", "staking",
        "caller", evm.Caller.Hex(),
    )

    logger.Debug("Precompile called", "input_size", len(input))

    defer func(start time.Time) {
        logger.Debug("Precompile completed",
            "duration", time.Since(start),
            "gas_used", evm.StateDB.GetGasUsed(),
        )
    }(time.Now())

    // 실행 로직...
}
```

9.2 메트릭스

```go
var (
    precompileCallsTotal = promauto.NewCounterVec(
        prometheus.CounterOpts{
            Name: "evm_precompile_calls_total",
            Help: "Total number of precompile calls",
        },
        []string{"precompile", "method", "status"},
    )

    precompileDuration = promauto.NewHistogramVec(
        prometheus.HistogramOpts{
            Name: "evm_precompile_duration_seconds",
            Help: "Precompile execution duration",
        },
        []string{"precompile", "method"},
    )
)

func (p *stakingPrecompile) Run(input []byte) ([]byte, error) {
    method := parseMethod(input)
    timer := prometheus.NewTimer(precompileDuration.WithLabelValues("staking", method))
    defer timer.ObserveDuration()

    result, err := p.execute(input)

    status := "success"
    if err != nil {
        status = "error"
    }
    precompileCallsTotal.WithLabelValues("staking", method, status).Inc()

    return result, err
}
```

---

10. Best Practices

10.1 설계 원칙

1. **단일 책임**: 각 precompile은 하나의 명확한 기능만 담당
2. **최소 권한**: 필요한 최소한의 Keeper만 접근
3. **상태 격리**: Precompile 간 상태 공유 최소화
4. **명확한 에러**: 구체적인 에러 메시지 반환

10.2 Performance Tips

```go
// ✅ Good: 데이터 복사 최소화
func (p *precompile) Run(input []byte) ([]byte, error) {
    // Zero-copy slice
    data := input[4:] // No allocation
    return processData(data)
}

// ❌ Bad: 불필요한 복사
func (p *precompile) Run(input []byte) ([]byte, error) {
    data := make([]byte, len(input)-4)
    copy(data, input[4:]) // Unnecessary allocation
    return processData(data)
}
```

10.3 ABI 설계

```
// ✅ Good: 명확한 인터페이스
interface IStaking {
    function delegate(address validator, uint256 amount)
        external
        returns (bool);

    function getDelegation(address delegator, address validator)
        external
        view
        returns (uint256);
}

// ❌ Bad: 불명확한 인터페이스
interface IStakingBad {
    function call(bytes memory data) external returns (bytes memory);
}
```

10.4 체크리스트

- [ ] Input 검증 구현
- [ ] Gas 비용 벤치마크 완료
- [ ] 단위 테스트 작성 (커버리지 > 80%)
- [ ] 통합 테스트 (Solidity) 작성
- [ ] 재진입 공격 검토
- [ ] 정수 오버플로우 검토
- [ ] 에러 처리 검토
- [ ] 로깅 및 메트릭스 추가
- [ ] 문서화 (ABI, 사용 예제)
- [ ] 코드 리뷰 완료

참고 자료

공식 문서

- EVM Precompiled Contracts
- Ethereum Yellow Paper
- Cosmos SDK Documentation

구현 예제

- go-ethereum precompiles
- Evmos EVM Extensions
- Cosmos EVM

추가 읽을거리

- EIP-2003: EVMC
- EIP-2537: BLS12-381 Precompiles
- Stateless Ethereum Precompiles

다음 단계

다음 문서에서는 **CometBFT Consensus**를 다룹니다:

- Tendermint/CometBFT 합의 알고리즘 심층 분석
- Proposer selection, Voting, Commit
- Safety와 Liveness 증명
- 네트워크 파티션 대응

→ 09-cometbft-consensus.md